

2장 Props와 State 이해

이 장에서는 React 애플리케이션의 핵심 데이터 전달 메커니즘인 **props**를 깊이 이해하고, 상위 컴포넌트와 하위 컴포넌트 간의 구조적 관계 속에서 데이터를 주고받는 방식을 실습하며, props의 읽기 전용 성질과 타입 검증, drilling 문제까지 살펴봄으로써 재사용 가능한 컴포넌트를 설계하고 유지보수가 용이한 UI 구조를 구축하는 데 필요한 기초를 다진다.

2.1 Props 개념과 데이터 전달 구조

Props는 React에서 부모 컴포넌트가 자식 컴포넌트로 데이터를 전달하기 위한 읽기 전용 매개체로, 컴포넌트 간 데이터 흐름을 단방향으로 유지하여 예측 가능한 UI를 만들 수 있게 하며, 기본값 설정과 타입 검증 기능을 통해 코드의 안정성을 강화하고, 데이터 전달 과정에서 발생하는 props drilling 문제의 개념까지 이해하게 한다.

(1) Props의 정의와 특징

Props는 부모 컴포넌트가 자식 컴포넌트로 값을 전달할 때 사용하는 읽기 전용 속성으로, 불변성을 유지하며 단방향 데이터 흐름을 보장해 컴포넌트 간 역할을 명확히 구분하고 예측 가능한 UI를 만드는 핵심 요소이다.

① Props란

Props(속성)는 부모 컴포넌트가 자식 컴포넌트로 전달하는 읽기 전용 데이터 컨테이너입니다. 값은 문자열·숫자·불리언·배열·객체·함수 등 자바스크립트의 모든 값이 될 수 있습니다.

② 함수형 컴포넌트

함수형 컴포넌트에서는 props를 매개변수의 `props` 또는 구조분해 할당 방식으로 사용합니다.

```
function Greeting({ name, age }) {  
  return <div>{name} — {age}</div>;  
}
```

③ 클래스형 컴포넌트

클래스형 컴포넌트에서는 `this.props` 처럼 객체의 속성으로 사용합니다.

```
class Greeting extends React.Component {  
  render() {  
    return <div>{this.props.name}</div>;  
  }  
}
```

④ Props의 역할과 특징

- **읽기 전용(immutable by contract):** 자식은 props를 직접 수정하면 안 됨(예외적으로 로컬 복사본을 만들어 수정 가능).
- **단방향 데이터 흐름:** 상위 → 하위로만 전달되어 데이터 흐름이 예측 가능.
- **재사용성:** 동일 컴포넌트를 다양한 props로 재활용 가능.

(2) 부모 → 자식 데이터 전달 방식

부모 컴포넌트는 JSX 내 자식 태그에 속성 형태로 값을 전달하고, 자식은 props 객체를 통해 해당 데이터를 접근·사용하여 UI를 구성하며, 이는 상위 데이터가 하위에 안전하게 전파되는 표준적인 방식이다.

① 기본 방식

기본 방식은 JSX 속성으로 전달한다.

```
// 부모
<UserCard name="Kim" score={42} onClick={() => console.log('clicked')} />

// 자식
function UserCard({ name, score, onClick }) {
  return <div onClick={onClick}>{name} — {score}</div>;
}
```

② children 전달

태그 사이의 콘텐츠는 `props.children` 으로 전달된다.

```
<Card>
  <h3>Title</h3>
  <p>Body</p>
</Card>
```

③ 객체/함수 전달 유의사항

- 객체를 `{ user }` 형태로 전달할 때는 참조 안정성에 유의. 부모 렌더마다 `user` 참조가 바뀌면 자식도 재렌더.
 - 이벤트 핸들러는 보통 부모의 상태 변경 로직을 위임하는 형태로 전달한다(`onChange`, `onSelect` 등).
- 패턴: "값(value) + 콜백(handler)" 패턴으로 부모가 소유한 상태를 자식이 읽고 변경을 요청하는 구조가 일반적이다.

(3) 읽기 전용 데이터 흐름 원칙

Props는 하위 컴포넌트에서 직접 수정할 수 없으며, 오직 상위 컴포넌트에서만 변경 가능하도록 설계되어 데이터 불변성을 유지하고, 이로써 UI의 일관성과 애플리케이션 예측 가능성을 보장한다.

① 원칙

props는 자식에서 직접 변경 불가 — 데이터 변경은 항상 소유자(owner)인 상위 컴포넌트에서만 수행되어야 합니다.

② 패턴: Lifting State Up(상태 끌어올리기)

- 자식이 값을 바꿔야 할 때는 부모가 상태(state)를 가지고, 부모는 `value` 와 `set_value` (또는 `onChange`)를 props로 전달합니다.

```
function Parent() {
  const [text, setText] = useState("");
  return <Child value={text} onChange={v => setText(v)} />;
}

function Child({ value, onChange }) {
  return <input value={value} onChange={e => onChange(e.target.value)} />;
}
```

이유: 단일 출처(Single Source of Truth)가 UI 일관성·디버깅을 용이하게 합니다.

예외: 로컬 UI 상태(포커스, 입력중의 임시값 등)는 자식 컴포넌트 내부 상태로 유지 가능.

(4) props 기본값 설정과 타입 검증

React는 props가 전달되지 않았을 때의 안전성을 확보하기 위해 defaultProps를 활용한 기본값 지정과 PropTypes를 이용한 타입 검증을 제공하여 코드의 안정성과 유지보수성을 강화한다.

① defaultProps (과거 방식)

컴포넌트에 전달되지 않은 props에 기본값을 설정.

```
MyComp.defaultProps = { size: 'medium' };
```

- 함수형 컴포넌트에서는 **ES6 기본 매개변수**(`function Comp({ size = 'medium' })`)를 더 자주 사용합니다.

② PropTypes (런타임 검증)

```
import PropTypes from 'prop-types';
```

```
MyComp.propTypes = {  
  title: PropTypes.string.isRequired,  
  count: PropTypes.number,  
  onClick: PropTypes.func,  
};
```

- PropTypes는 개발 모드에서 런타임 경고를 제공해 잘못된 prop 사용을 조기 발견하게 해줍니다.

TypeScript 대안: 정적 타입 검사에 TypeScript(인터페이스/타입)를 사용하면 컴파일 타임에 타입 안정성을 확보할 수 있습니다.

```
type Props = { title: string; count?: number };  
const MyComp: React.FC<Props> = ({ title, count = 0 }) => { ... };
```

(5) props drilling 문제와 한계

여러 계층의 컴포넌트를 거쳐 동일한 props를 전달해야 하는 경우, 불필요하게 중간 컴포넌트까지 데이터가 퍼지는 props drilling 문제가 발생하며, 이는 Context API나 전역 상태 관리 도구로 보완할 수 있다.

① 문제 정의

깊은 컴포넌트 트리에서 상위의 데이터/함수를 중간의 여러 컴포넌트가 실제로 사용하지 않음에도 전달해야 하는 상황이 반복되면 코드가 복잡해지고 유지보수성이 떨어집니다.

```
App → A → B → C → D (D가 token 필요) // A,B,C가 token을 사용하지 않더라도 전달해야 함
```

② 해결 전략

- Context API:** 전역적이면서 트리의 깊이에 상관없이 값을 공급할 수 있음.

```
const AuthContext = createContext(null);  
function App() {  
  const auth = useProvideAuth();  
  return <AuthContext.Provider value={auth}><Tree /></AuthContext.Provider>;  
}  
function DeepChild() {  
  const auth = useContext(AuthContext);
```

```
// auth 사용
}
```

단점: 자주 변경되는 값을 Context로 두면 전체 컨슈머가 재렌더될 수 있음
(성능 고려)

- **전역 상태관리 라이브러리**: Redux, Zustand, Recoil 등 — 복잡한 상태/비동기 로직을 중앙에서 관리
- **컴포지션/포털/콜백 전달 전략**: 필요한 데이터만 하위로 넘기고, 중간 컴포넌트는 노출하지 않는 설계(컴포넌트 경계 재설계)
- **부분적 Context 분할**: 여러 Context로 나누어 값 변경 시 영향을 줄임

2.2 상위/하위 컴포넌트 관계 실습

상위 컴포넌트에서 props를 통해 데이터를 자식에게 전달하고, 자식 컴포넌트는 이를 받아 UI에 반영하는 구조적 흐름을 실습하며, 중첩된 컴포넌트 구조 속에서 props 전달 패턴을 학습하고, 이벤트 핸들러를 props로 주입하는 기법까지 익혀 재사용 가능하고 유연한 UI 컴포넌트를 구현할 수 있다.

Vite 기반 React 프로젝트 생성

```
npm create vite@latest ex02
```

- 명령 실행 후 프로젝트 이름: `ch02`
- 프레임워크 선택: `React`
- Variant 선택: `JavaScript` 또는 `TypeScript`
(여기서는 JavaScript 기준)

애플리케이션 디렉터리 구조

```
ex02/
├─ package.json
├─ vite.config.js
├─ index.html
├─ public/
├─ src/
│  ├─ main.jsx
│  ├─ App.jsx
│  ├─ components/
│  │  ├─ ParentComp1.jsx
│  │  ├─ ChildrenComp1.jsx
│  │  ├─ ParentComp2.jsx
│  │  ├─ ChildrenComp2.jsx
│  │  ├─ ParentComp3.jsx
│  │  ├─ ChildrenComp3.jsx
│  │  ├─ ParentComp4.jsx
│  │  ├─ ChildrenComp4.jsx
│  │  ├─ ParentComp5.jsx
│  │  └─ ChildrenComp5.jsx
│  └─ styles/
│      └─ App.css
└─ README.md
```

디렉터리 이동 및 의존성 라이브러리 설치

```
cd ch02      # 디렉터리 이동
npm install   # 의존성 라이브러리 설치
```

- React 19, React DOM 19 포함
- ESLint, Prettier, 기타 개발용 패키지 설치 가능

개발 서버 실행

```
npm run dev
```

- 터미널에 표시되는 URL 확인 후 브라우저에서 접속
예: <http://localhost:5173>

(1) 부모 컴포넌트에서 자식 컴포넌트로 props 전달하기

부모 컴포넌트는 JSX 태그에 속성처럼 값을 지정하여 자식에게 props를 전달하며, 이는 문자열, 숫자, 객체, 함수 등 다양한 형태로 넘길 수 있어 유연한 UI 구성을 가능하게 한다.

- 부모 컴포넌트는 JSX 속성을 통해 데이터를 전달하며, 문자열·숫자·불리언·배열·객체·함수·JSX 등 모든 자바스크립트 값을 전달 가능
- children을 통해 JSX 조각을 전달하면 자식 컴포넌트가 해당 내용을 렌더링 가능.
- 객체나 함수 전달 시 참조 안정성을 고려하지 않으면 매 렌더마다 새 객체/함수 생성으로 자식이 불필요하게 재렌더될 수 있음
- 값(value) + 콜백(handler) 패턴으로 상위 상태를 안전하게 관리하고, 필요시 `useMemo` / `useCallback` 으로 최적화
- prop spreading(`{...props}`)은 간결하지만 의도치 않은 prop 유출이나 덮어쓰기 위험이 있으므로 주의

① 기본 값 전달

부모는 JSX 속성 형태로 문자열, 숫자, 불리언 등 원시값을 전달하고 자식은 props에서 바로 읽어 렌더링하며, 기본값 선언과 명확한 prop 이름으로 가독성과 안전성을 확보한다.

② 객체/배열 전달과 참조 안정성

객체나 배열을 props로 전달할 때는 매 렌더마다 새 참조가 생성되지 않도록 `useMemo` 를 사용하거나 필요한 필드만 전달해 불필요한 자식 재렌더를 방지하고 성능을 최적화한다.

③ 함수/이벤트 핸들러 전달

부모에서 상태 변경이나 이벤트 핸들러를 props로 전달하면 자식은 호출만으로 상위 로직을 실행할 수 있으며, `useCallback` 으로 참조를 고정해 성능과 일관성을 유지한다.

④ children과 render-prop 전달

JSX 조각이나 렌더 로직을 `children` 또는 render-prop 패턴으로 전달하면 재사용성과 유연성을 높일 수 있으며, API 일관성과 과도한 콜백 전달로 인한 복잡성을 주의해야 한다.

src/components/ParentComp1.jsx

```
import React, { useCallback } from 'react';
import ChildrenComp1 from './ChildrenComp1';
```

```
export default function ParentComp1() {
  const handleClick = useCallback(() => alert('버튼 클릭!'), []);
  const user = { id: 1, name: 'Kim' };

  return (
    <div className="section">
      <h2>ParentComp1: 기본 props 전달</h2>
      <ChildrenComp1 name="Alice"
        age={25}
        user={user}
        onClick={handleClick}
      >
        <strong>자식 children 전달</strong>
      </ChildrenComp1>
    </div>
  );
}
```

src/components/ChildrenComp1.jsx

```
import React from 'react';

export default function ChildrenComp1({ name, age, user, onClick, children }) {
  return (
    <div>
      <p>이름: {name}</p>
      <p>나이: {age}</p>
      <p>유저 객체: {user.name}</p>
      <button onClick={onClick}>클릭</button>
      <div>{children}</div>
    </div>
  );
}
```

(2) 자식 컴포넌트에서 props 수신 및 렌더링

자식 컴포넌트는 props 매개변수나 구조분해 할당을 통해 데이터를 수신하고, 해당 값을 JSX 내부에서 렌더링하여 부모가 전달한 데이터를 시각적으로 반영한다.

- 자식 컴포넌트는 props 매개변수 또는 `this.props` 로 전달된 데이터를 수신.
- 구조분해 할당을 통해 가독성 높이기 및 기본값 선언 가능.
- 조건부 렌더링과 옵셔널 체이닝(`?.`)으로 `undefined` 값 안전하게 처리.
- prop 변경에 따른 사이드이펙트는 `useEffect` 의존성 배열에 prop을 넣어 관리.
- 불필요 렌더 방지를 위해 `React.memo` 나 맞춤형 비교함수를 적용

① 구조분해 할당과 기본값

자식 컴포넌트는 props를 구조분해 할당해 가독성을 높이고, ES6 기본 매개변수로 기본값을 지정하며, TypeScript 또는 PropTypes로 타입을 문서화해 안전한 렌더링을 보장한다.

② 조건부 렌더링과 안전한 접근

props가 없는 경우나 일부 필드가 undefined일 때는 삼항 연산자나 `&&` 연산, 옵셔널 체이닝을 사용해 안전하게 렌더링하고 `false` 값과 `undefined`의 의미를 명확히 구분해야 UI 오류를 예방한다.

③ prop 변경에 따른 side-effect 처리

자식 컴포넌트가 prop 값 변경 시 수행할 부수 효과는 `useEffect`의 의존성 배열에 해당 prop을 넣어 실행하고, 필요 시 정리(cleanup)를 통해 메모리 누수나 이벤트 누수를 방지한다.

④ 렌더 최적화와 React.memo

정적이거나 변경이 드문 props를 받는 자식 컴포넌트는 `React.memo`를 사용해 불필요한 재렌더를 방지하며, 복잡한 비교가 필요하면 맞춤형 비교함수로 정확성과 성능을 동시에 확보한다.

src/components/ParentComp2.jsx

```
import React from 'react';
import ChildrenComp2 from './ChildrenComp2';

export default function ParentComp2() {
  return (
    <div className="section">
      <h2>ParentComp2: props 수신 및 렌더링</h2>
      <ChildrenComp2 name="Bob" age={30} />
    </div>
  );
}
```

src/components/ChildrenComp2.jsx

```
import React, { useEffect } from 'react';

export default React.memo(function ChildrenComp2({ name = '무명', age = 0 }) {
  useEffect(() => {
    console.log('props 변경 감지: ${name}, ${age}');
  }, [name, age]);

  return (
    <div>
      <p>이름: {name}</p>
      <p>나이: {age}</p>
    </div>
  );
});
```

(3) 중첩 구조에서 props 전달 패턴 실습

컴포넌트가 여러 단계로 중첩될 경우, props는 상위에서 하위로 단계별로 전달되며, 실습을 통해 데이터가 깊은 계층으로 어떻게 전파되는지 확인할 수 있다.

- 깊은 컴포넌트 트리에서 상위 데이터가 하위까지 전달되어야 하는 경우 props drilling 문제가 발생
- 해결 패턴: 단계적 전달, Composition/children, Context API, 전역 상태(Redux/Zustand), forwardRef/Portal
- Context를 활용하면 트리 깊이와 상관없이 필요한 컴포넌트에서 데이터 접근 가능

src/components/ParentComp3.jsx

```
import React from 'react';
import ChildrenComp3 from './ChildrenComp3';

export default function ParentComp3() {
  const message = "깊은 트리 데이터 전달";

  return (
    <div className="section">
      <h2>ParentComp3: 중첩 구조 props 전달</h2>
      <ChildrenComp3 message={message} />
    </div>
  );
}
```

src/components/ChildrenComp3.jsx

```
import React from 'react';

export default function ChildrenComp3({ message }) {
  return (
    <div>
      <p>자식 컴포넌트: {message}</p>
      <div>
        <p>중첩 자식에서도 동일 메시지 사용 가능</p>
      </div>
    </div>
  );
}
```

(4) 이벤트 핸들러를 props로 전달하는 방법

부모는 함수 형태의 이벤트 핸들러를 props로 전달하고, 자식은 이를 호출하여 부모의 상태를 변경하거나 상위 로직을 실행할 수 있어 양방향 상호작용이 가능하다.

- 부모의 상태 변경이나 로직을 자식 이벤트에서 트리거하도록 props로 함수 전달
- 인자 전달, `preventDefault`, `SyntheticEvent` 이해 필수
- `useCallback` 으로 참조 고정, Controlled 컴포넌트와 `value/onChange` 패턴 병행

src/components/ParentComp4.jsx

```
import React, { useState, useCallback } from 'react';
import ChildrenComp4 from './ChildrenComp4';

export default function ParentComp4() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => setCount(prev => prev + 1), []);

  return (
    <div className="section">
      <h2>ParentComp4: 이벤트 핸들러 전달</h2>
      <p>카운트: {count}</p>
    </div>
  );
}
```



```

    <ChildrenComp4 onIncrement={increment} />
  </div>
);
}

```

src/components/ChildrenComp4.jsx

```

import React from 'react';

export default function ChildrenComp4({ onIncrement }) {
  return <button onClick={onIncrement}>증가</button>;
}

```

(5) props를 활용한 UI 재사용 사례

동일한 컴포넌트에 props로 다른 데이터를 전달하면, 하나의 코드로 여러 형태의 UI를 구성할 수 있어 코드 중복을 줄이고 재사용성을 극대화할 수 있다.

- 동일 컴포넌트를 다양한 상황에서 재사용 가능
- variant, size, disabled, as 등의 prop으로 동작·스타일·태그 제어
- Storybook을 활용해 사례별 문서화 및 테스트, 접근성(aria) 고려

src/components/ParentComp5.jsx

```

import React from 'react';
import ChildrenComp5 from './ChildrenComp5';

export default function ParentComp5() {
  return (
    <div className="section">
      <h2>ParentComp5: props로 UI 재사용</h2>
      <ChildrenComp5 variant="primary" size="lg">Primary Large</ChildrenComp5>
      <ChildrenComp5 variant="secondary" size="sm">Secondary Small</ChildrenComp5>
    </div>
  );
}

```

src/components/ChildrenComp5.jsx

```

import React from 'react';

export default function ChildrenComp5({ variant = 'primary', size = 'md', children }) {
  const className = `btn ${variant} ${size}`;
  return <button className={className}>{children}</button>;
}

```

(6) 애플리케이션 파일 작성

Vite 애플리케이션 파일이란? main.jsx, App.jsx, App.css 등의 애플리케이션의 실행시 브라우저의 렌더링에 필요한 파일들로 App.js는 여러 컴포넌트를 통합하고, 렌더링되는 컴포넌트를 제어합니다. main.jsx는 여러 컴포넌트를 App.js에서 조합해 놓으면, 그 내용을 index.html의 id가 App인 곳에 내보내 웹 브라우저에 출력되게 합니다.

src/App.jsx

```
import React from 'react';
import ParentComp1 from './components/ParentComp1';
import ParentComp2 from './components/ParentComp2';
import ParentComp3 from './components/ParentComp3';
import ParentComp4 from './components/ParentComp4';
import ParentComp5 from './components/ParentComp5';

export default function App() {
  return (
    <div className="app">
      <h1>2.2 상위/하위 컴포넌트 관계 실습 - ch02</h1>
      <ParentComp1 />
      <ParentComp2 />
      <ParentComp3 />
      <ParentComp4 />
      <ParentComp5 />
    </div>
  );
}
```

src/styles/App.css

```
.app { font-family: Arial, sans-serif; padding: 20px; }
.section { margin-bottom: 30px; padding: 10px; border: 1px solid #ccc; }
.btn { padding: 5px 10px; margin: 5px; cursor: pointer; }
.btn.primary { background-color: #007bff; color: white; }
.btn.secondary { background-color: #6c757d; color: white; }
.btn.sm { font-size: 12px; }
.btn.md { font-size: 16px; }
.btn.lg { font-size: 20px; }
```

src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './styles/App.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

2.3 State 개념 및 활용

React의 State는 컴포넌트 내부에서 관리되는 동적 데이터로, 사용자의 상호작용이나 이벤트 발생 시 변경되면 해당 컴포넌트를 자동으로 재렌더링하여 UI를 최신 상태로 유지하며, props와 달리 컴포넌트 자체가 직접 관리하고 변경할 수 있는 특성을 가지며, 배열·객체·불리언 등 다양한 형태의 데이터를 다루면서 Controlled Component, 조건부 렌더링, 불변성 패턴 등과 결합해 효율적이고 예측 가능한 UI 상태 관리를 구현할 수 있다.

애플리케이션 생성

```
# 1. CRA 프로젝트 생성과 디렉터리 이동
npx create-react-app ex03
cd ex03

# 2. React 19로 업그레이드
npm install react@19 react-dom@19

# 3. 프로젝트 실행
npm start
```

- 브라우저에서 `http://localhost:3000` 접속

프로젝트 디렉터리 구조

```
ex03/
├─ package.json
├─ README.md
├─ public/
│  └─ index.html
│  └─ ...
└─ src/
   ├─ index.js
   ├─ App.js
   ├─ App.css
   └─ components/
      ├─ StateComp1.js # (1) State 정의
      ├─ Counter.js   # (2) State 선언 및 변경
      ├─ StateComp2.js # (2) State 선언 및 변경
      ├─ StateComp3.js # (3) State 변경과 UI 갱신
      ├─ StateComp4.js # (4) State 활용 패턴
      └─ StateComp5.js # (5) State와 Props 비교
   └─ styles/
      └─ App.css
```

(1) State 정의

State는 React 컴포넌트 내부에서 관리되는 데이터로, UI의 동적 변화를 가능하게 하며, 함수형 컴포넌트에서는 `useState` Hook, 클래스형 컴포넌트에서는 `this.state`와 `this.setState`를 통해 선언하고 조작할 수 있어 컴포넌트의 로컬 상태를 안전하고 예측 가능하게 관리할 수 있다.

- React 컴포넌트 내부에서 관리되는 데이터
- UI를 동적으로 갱신할 때 사용
- 함수형 컴포넌트에서는 `useState` Hook으로 선언
- 클래스형 컴포넌트에서는 `this.state`와 `this.setState()` 사용

src/components/StateComp1.js

```
import React from 'react';

export default function StateComp1() {
  // State 정의 설명
  return (
    <div className="section">
      <h2>(1) State 정의</h2>
      <p>State는 컴포넌트 내부에서 관리되는 데이터로, UI를 동적으로 갱신할 때 사용됩니다.</p>
    </div>
  );
}
```

(2) State 선언 및 변경 원리

함수형 컴포넌트에서는 useState를 사용해 초기값과 setter 함수를 정의하고, setter를 통해 값을 변경하면 컴포넌트가 재렌더링되며, 직접 state를 수정하지 않고 항상 불변성을 유지하여 안정적 UI 갱신과 참조 일관성을 확보할 수 있으며, 이벤트 핸들러와 결합해 사용자 상호작용에 따른 동적 업데이트가 가능하다.

src/components/Counter.js

```
import React, { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0); // 초기값 0

  const increment = () => setCount(count + 1);

  return (
    <div>
      <p>카운트: {count}</p>
      <button onClick={increment}>증가</button>
    </div>
  );
}
```

- `useState(초기값)` → `[state값, setState함수]` 반환
- `setState` 호출 시 컴포넌트 재렌더링
- 불변성 유지: 직접 state 수정 금지, 항상 setter 사용

src/components/StateComp2.js

```
import React, { useState } from 'react';

export default function StateComp2() {
  const [count, setCount] = useState(0); // State 선언

  return (
    <div className="section">
      <h2>(2) State 선언 및 변경</h2>
      <p>카운트: {count}</p>
      <button onClick={() => setCount(count + 1)}>증가</button>
    </div>
  );
}
```

```
};  
}
```

(3) State 변경과 UI 갱신

State가 변경되면 React가 Virtual DOM과 실제 DOM을 비교해 필요한 부분만 갱신하며, 버튼 클릭, 입력값 변경, API 응답 등 이벤트를 통해 setter를 호출하면 해당 컴포넌트와 하위 컴포넌트가 최신 상태로 재렌더링되며, 이를 통해 UI가 항상 state와 동기화된 상태로 유지된다.

- 버튼 클릭, 입력값 변화 등 이벤트 발생
- `setState` 호출로 state 변경
- React가 변경된 state를 감지
- Virtual DOM 비교 후 필요한 부분만 실제 DOM 갱신
- UI에 최신 상태 반영

src/components/StateComp3.js

```
import React, { useState } from 'react';  
  
export default function StateComp3() {  
  const [text, setText] = useState("");  
  
  return (  
    <div className="section">  
      <h2>(3) State 변경과 UI 갱신</h2>  
      <input type="text"  
        value={text}  
        onChange={(e) => setText(e.target.value)}  
        placeholder="입력하세요"  
      />  
      <p>입력값: {text}</p>  
    </div>  
  );  
}
```

(4) State 활용 상태 기반 동적 화면 구현

State는 Controlled Component 패턴으로 Form 입력을 관리하거나, 조건부 렌더링을 통해 UI 표시 여부를 제어하며, 배열과 객체 상태는 불변성을 유지하며 spread 연산자나 setter 함수를 통해 안전하게 업데이트하고, 이로써 동적 데이터와 UI를 일관성 있게 연결하고 재사용 가능한 패턴을 구현할 수 있다.

- **Form 입력 관리:** Controlled Component (`value` + `onChange`)
- **조건부 렌더링:** `isVisible ? <Component /> : null`
- **배열/객체 상태:** 불변성 유지하며 업데이트 (`...spread`)

```
const [items, setItems] = useState([]);  
setItems(prev => [...prev, newItem]);
```

src/components/StateComp4.js

```
import React, { useState } from 'react';
```

```
export default function StateComp4() {
  const [items, setItems] = useState([]);

  const addItem = () => {
    setItems(prev => [...prev, `Item ${prev.length + 1}`]);
  };

  return (
    <div className="section">
      <h2>(4) State 활용 패턴</h2>
      <button onClick={addItem}>아이템 추가</button>
      <ul>
        {items.map((item, idx) => <li key={idx}>{item}</li>)}
      </ul>
    </div>
  );
}
```

(5) State와 Props 비교

State는 컴포넌트 내부에서 관리하며 직접 변경 가능한 로컬 상태 데이터인 반면, props는 부모가 전달하고 자식이 읽기만 가능한 데이터로, State는 UI 동적 갱신과 사용자 상호작용에 사용되고 props는 계층 구조를 통한 데이터 전달에 사용되며, 두 개념을 함께 활용해 컴포넌트 간 데이터 흐름과 로컬 상태를 효율적으로 관리할 수 있다.

항목	Props	State
관리	부모 컴포넌트	자기 자신
변경	읽기 전용	컴포넌트 내부에서 변경 가능
용도	데이터 전달	UI 동적 변화, 사용자 상호작용

src/components/StateComp5.js

```
import React, { useState } from 'react';

function Child({ count }) {
  return <p>자식 컴포넌트에서 받은 count: {count}</p>;
}

export default function StateComp5() {
  const [count, setCount] = useState(0);

  return (
    <div className="section">
      <h2>(5) State와 Props 비교</h2>
      <p>부모 State: {count}</p>
      <button onClick={() => setCount(count + 1)}>증가</button>
      <Child count={count} />
      <p>State는 컴포넌트 내부에서 관리되고, Props는 부모에서 자식으로 전달됩니다.</p>
    </div>
  );
}
```

(6) 애플리케이션 파일 작성

CRA 애플리케이션 파일이란? index.js, App.js, App.css 등의 애플리케이션의 실행시 브라우저의 렌더링에 필요한 파일들로 App.js는 여러 컴포넌트를 통합하고, 렌더링되는 컴포넌트를 제어합니다. main.jsx는 여러 컴포넌트를 App.js에서 조합해 놓으면, 그 내용을 index.html의 id가 App인 곳에 내보내 웹 브라우저에 출력되게 합니다.

src/index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import './App.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

src/App.js

```
import React from 'react';
import StateComp1 from './components/StateComp1';
import StateComp2 from './components/StateComp2';
import StateComp3 from './components/StateComp3';
import StateComp4 from './components/StateComp4';
import StateComp5 from './components/StateComp5';

export default function App() {
  return (
    <div className="app">
      <h1>2.3 State 개념 및 활용 - ex03</h1>
      <StateComp1 />
      <StateComp2 />
      <StateComp3 />
      <StateComp4 />
      <StateComp5 />
    </div>
  );
}
```

src/App.css

```
.app { font-family: Arial, sans-serif; padding: 20px; }
.section { margin-bottom: 30px; padding: 10px; border: 1px solid #ccc; }
button { margin-top: 5px; padding: 5px 10px; cursor: pointer; }
input { margin-top: 5px; padding: 5px; }
ul { margin-top: 5px; }
```

2.4 컴포넌트 설계 원칙

컴포넌트 설계 원칙은 React 애플리케이션에서 유지보수성과 재사용성을 높이기 위해 UI 요소를 단일 책임과 역할 기반으로 분할하고, Props와 이벤트를 활용한 범용 구조를 설계하며, 폴더 구조와 파일 명명 규칙을 체계적으로 정립하고, 도메인 중심, 기능 중심, 혼합형 등 실무 관점 설계 사례를 비교하여 팀 협업과 테스트, 확장성까지 고려한 최적의 설계 전략을 제시하는 중요한 가이드라인이다.

(1) UI 요소의 컴포넌트 분할 기준

UI 요소의 컴포넌트 분할 기준은 단일 책임과 상태/로직 분리를 기반으로 반복 요소를 별도 컴포넌트로 추출하고, 부모-자식 계층 구조를 명확히 정의하여 Prop 전달과 상태 관리가 효율적이며 유지보수와 확장성이 높은 구조를 만드는 데 목적이 있다.

① 단일 책임 원칙 적용

각 컴포넌트는 하나의 기능 또는 UI 블록만 담당하며, 화면 영역과 기능 단위로 명확히 구분해 코드 가독성과 유지보수성을 높인다.

- **정의:** 한 컴포넌트는 하나의 역할 또는 UI 블록만 담당합니다.
- **목적:** 코드 가독성과 유지보수성을 높이고, 테스트 용이성을 확보합니다.
- **실무 예시:**
 - 버튼, 카드, 입력 필드 등 독립 UI는 개별 컴포넌트로 분리
 - 화면 전체를 단일 컴포넌트로 작성하지 않고, 영역별/기능별로 나누어 관리

```
// ButtonComp.jsx
export default function ButtonComp({ label, onClick }) {
  return <button onClick={onClick}>{label}</button>;
}
```

② 상태와 로직 분리

컨테이너와 프레젠테이션 패턴을 적용해 상태와 이벤트는 상위 컴포넌트가 관리하고 렌더링은 하위 컴포넌트가 수행하도록 구조화한다.

- **정의:** 상태(State)와 비즈니스 로직은 컨테이너(상위) 컴포넌트에서 관리하고, UI 렌더링은 프레젠테이션(하위) 컴포넌트에서 수행합니다.
- **목적:** 컴포넌트 재사용성을 높이고, 상태 변경이 UI 렌더링에 영향을 최소화합니다.
- **패턴:** Container vs Presentational
 - Container: 상태 관리, API 호출, 이벤트 처리
 - Presentational: UI 렌더링, Props 기반

```
// ContainerComp.jsx
import { useState } from "react";
import ButtonComp from "./ButtonComp";

export default function ContainerComp() {
  const [count, setCount] = useState(0);
  return <ButtonComp label={`Clicked ${count}`} onClick={() => setCount(count + 1)} />;
}
```

③ 반복 요소 분리

반복되는 Button, Card 등 UI 블록을 별도 컴포넌트로 추출하여 코드 중복을 제거하고 유지보수를 용이하게 만든다.

- **정의:** 반복되는 UI 요소(Button, Card, List Item 등)를 별도의 컴포넌트로 추출합니다.
- **목적:** 코드 중복 제거, 유지보수 용이, 재사용성 확보
- 동일한 카드 UI가 여러 화면에 존재할 때 `CardComp.jsx` 로 분리


```
// CardComp.jsx
export default function CardComp({ title, description }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <p>{description}</p>
    </div>
  );
}
```

- 반복 렌더링: 배열 map()과 함께 사용

```
const items = [{title:"A", description:"AA"}, {title:"B", description:"BB"}];
items.map((item, idx) => <CardComp key={idx} {...item} />);
```

④ 컴포넌트 계층 구조 정의

부모 컴포넌트가 상태를 관리하고 자식은 UI만 렌더링하며 Prop 전달 계층을 명확히 설계하여 컴포넌트 간 역할을 구분한다.

- **정의:** 부모 컴포넌트가 상태를 관리하고 자식은 UI만 렌더링하도록 계층 구조를 명확히 합니다.
- **목적:** Prop 전달 계층을 명확히 하여 데이터 흐름을 이해하기 쉽고, 디버깅과 테스트가 용이

```
// ParentComp.jsx
import ChildComp from "./ChildComp";
export default function ParentComp() {
  const [value, setValue] = useState("");
  return <ChildComp value={value} onChange={e => setValue(e.target.value)} />;
}

// ChildComp.jsx
export default function ChildComp({ value, onChange }) {
  return <input type="text" value={value} onChange={onChange} />;
}
```

- 상위에서 상태 관리 → 하위는 Props 기반 UI 렌더링
- Prop 계층이 깊어질 경우 Context API나 Composition 패턴 활용

(2) 재사용 가능한 컴포넌트 구조 설계

재사용 가능한 컴포넌트 구조 설계는 Props와 이벤트를 기반으로 다양한 UI 변형을 지원하고, 일관된 스타일과 레이아웃을 유지하며, 범용 컴포넌트를 추상화하고 Composition 패턴을 활용해 Context나 전역 상태와 연계 가능한 유연한 구조를 만드는 것을 목표로 한다.

① Props와 이벤트 기반 구성

Props와 이벤트 핸들러를 활용해 컴포넌트를 다양한 상황에서 재사용 가능하도록 설계하고 상위 상태와 연동한다.

- **값(Value) + 콜백(Handler) 패턴:** 입력/제어형 컴포넌트는 `value` (또는 `checked`)와 `onChange` 를 쌍으로 제공. uncontrolled도 `defaultValue` 로 지원.
- **이벤트 네이밍:** `onSubmit`, `onSelect`, `onChange` 등 `onX` 형태로, 핸들러는 상위에서 `handleX` 로 작성.
- **불필요한 props 억제:** 컴포넌트가 관리할 props를 문서화(PropTypes / TS 타입). 지나친 props는 API 혼란 초래.
- **참조 안정성:** 부모에서 객체/함수 props를 넘길 때 `useMemo` / `useCallback` 으로 참조를 고정해 자식의 불필요한 재렌더 방지.
- **payload 설계:** 이벤트 콜백의 인자는 가능하면 단순(primitive 또는 명확한 객체 `{ id, value }`)으로 설계.

예: Button (JSX, JS)

```
// components/Button.jsx
export default function Button({ variant = 'primary', size = 'md', disabled = false, onClick, children, className="" })
{
  const cls = `btn ${variant} ${size} ${disabled ? 'is-disabled' : ''} ${className}`;
  return (
    <button className={cls} onClick={disabled ? undefined : onClick} aria-disabled={disabled}>
      {children}
    </button>
  );
}
```

예: Controlled Input pattern

```
function TextInput({ value, onChange, placeholder }) {
  return <input value={value} onChange={e => onChange(e.target.value)} placeholder={placeholder} />;
}
```

② 스타일/레이아웃 일관성 유지

CSS 모듈, styled-components, Tailwind 등을 사용해 spacing, typography, color system을 일관되게 적용한다.

- **Design tokens / 변수화:** spacing, color, font-size 등을 `:root` CSS 변수 또는 Tailwind/Tokens 파일로 관리.
- **스타일 방식 결정:** CSS Modules, styled-components, Tailwind 중 팀 컨벤션에 맞게 선택.
 - CSS Modules: 로컬 스코프, 빌드 도구 의존 적음.
 - styled-components: JS 기반 테마 확장(ThemeProvider) 강점.
 - Tailwind: 유틸리티 기반 빠른 개발, 일관된 spacing scale.
- **토론**

```
/* variables.css */
:root {
  --space-xs: 4px; --space-sm: 8px; --space-md: 16px; --space-lg: 24px;
  --color-primary: #0b74ff; --color-muted: #6b7280;
  --font-base: 16px;
}
```

- **Spacing & Typography 규칙:** 4px 또는 8px 단위로 스케일 통일, 폰트 스케일(16/14/12) 규정.
- **className 합치기:** `clsx` / `classnames` 사용 권장 (`className` prop 노출해 스타일 오버라이드 허용).

③ 범용 컴포넌트 추상화 수준 결정

Button, Input, Card 등 화면 독립적인 기본 컴포넌트를 설계하고 특정 화면에 종속되지 않도록 일반화한다.

- **언제 추상화할지:** 같은 UI가 2곳 이상 사용되고, 변형(variant/size/disabled)으로 대응 가능하면 기본 컴포넌트로 추출.
- **추상화 과하지 않게:** 너무 범용으로 만들면 API가 복잡해짐. 최소한의 옵션(variant/size/disabled/as)만 제공.
- **Polymorphic / as prop:** 태그를 바꿔야 할 때 `as` prop 제공 (e.g., `as="a"` 또는 `as={Link}`).
- **기능 분리:** `Button` 은 단순 클릭·상태·스타일, 복잡한 로직(서버 요청 등)은 상위에서 처리.

예: polymorphic Button (TypeScript 스타일)

```
type ButtonProps = {
  variant?: 'primary' | 'secondary';
  size?: 'sm' | 'md' | 'lg';
  as?: any; // 'button' | 'a' | Component
} & React.ComponentPropsWithoutRef<'button'>;
```

```
export function Button({ as: Component = 'button', variant='primary', size='md', ...rest }: ButtonProps) {
  const className = `btn ${variant} ${size} ${rest.className ?? ''}`;
  return <Component className={className} {...rest} />;
}
```

④ Composition 패턴 활용

children, render-prop, HOC 패턴을 적용하고 필요시 Context 또는 전역 상태와 연계해 유연성을 높인다.

- **children:** 가장 간단하고 직관적인 합성 수단. 레이아웃 컨테이너, 카드, 모달에 적합.

```
<Card>
  <Card.Header>타이틀</Card.Header>
  <Card.Body>본문</Card.Body>
</Card>
```

- **Render props:** 자식이 렌더 로직을 부모에게 위임해야 할 때 (`(props)=>JSX`).

```
<DataLoader url="/api">
  {{{ data, loading }}} => loading ? <Spinner/> : <List data={data}/>
</DataLoader>
```

- **HOC (Higher-Order Component):** 공통 로직 주입 (예: `withAuth`, `withI18n`). 단점: 컴포넌트 트리 가독성 저하 가능.
- **Custom Hooks:** 로직 재사용 권장(비즈니스/상태 로직을 훅으로 분리).
- **forwardRef:** 외부에서 focus 관리 등 DOM 접근이 필요하면 `forwardRef` 로 ref 전달 허용.
- **Context 조합:** 공통 설정(Theme, Auth)을 전역으로 공급. 빈번한 업데이트 값은 별도 Context로 분리해 재렌더 범위 최소화.

⑤ 접근성(Accessibility)와 API 설계

- **ARIA, keyboard handling:** 버튼의 role, 키보드 포커스, `aria-*` props 지원.
- **Disabled 처리:** `aria-disabled`, `tabIndex=-1` 고려.
- **Labeling:** 입력 컴포넌트는 `id` / `htmlFor` 규칙을 문서화.

⑥ 성능·테스트·문서화

- **성능:** 불필요한 재렌더 방지 (`React.memo`, `useCallback`, `useMemo`), 리스트 key 정책.
- **테스트:** 단위 테스트(동작), 스냅샷(구조), 접근성 테스트(axe).
- **문서화:** Storybook으로 각 variant/size/상태 케이스 문서화 및 시각적 확인. Storybook controls로 props 조작 제공.
- **버전·변경 관리:** 컴포넌트 API 변경 시 마이그레이션 문서 제공.

(3) props 설계 전략 및 폴더 구조

Props 설계 전략과 폴더 구조는 필수/선택 Props를 구분하고 타입을 정의하며, props drilling을 최소화하고 Context나 전역 상태와 연계하며, 도메인별·기능별 폴더 구조와 PascalCase 파일 명명, index 재export 구조를 통해 재사용성과 확장성을 고려한 체계적 구조를 구축하는 것이다.

① Props 설계 전략

필수와 선택 Props를 구분하고 DefaultProps 또는 TypeScript 타입을 정의해 안정적이고 예측 가능한 컴포넌트 동작을 보장한다.

- 간결하고 예측 가능한 API 제공

- 최소한의 props로 최대 표현(심플한 기본값)
- 안정적 타입(런타임/정적)으로 계약 문서화
- 퍼포먼스(참조 안정성)와 접근성 고려

핵심 규칙

1. 필수 vs 선택(props) 구분

- 필수는 명확히 `isRequired` (PropTypes) 또는 TypeScript `required` 로 표시.
- 선택 props에는 명시적 기본값 제공(ES6 기본 인자 또는 `defaultProps`).

2. 명확한 네이밍 규칙

- boolean: `isOpen`, `isDisabled`, `hasError` (접두사 is/has 권장)
- 이벤트: `onX` (예: `onChange`, `onSelect`) — 핸들러는 `handleX` 로 구현
- 상태: `value` / `defaultValue` (컨트롤드/언컨트롤드 구분)

3. Controlled vs Uncontrolled

- 폼 입력 등은 `value` + `onChange` (controlled)과 `defaultValue` (uncontrolled) 두 가지 패턴 지원 권장.
- 컴포넌트 코드: `const isControlled = value !== undefined;` 패턴으로 처리.

4. payload 설계(콜백 인자)

- 단순 컴포넌트는 `onChange(value)` 형태 권장(사용편의).
- 복잡한 컴포넌트는 `onChange({ id, value })` 처럼 명확한 Shape 사용.
- 브라우저 기본 이벤트를 그대로 전달할 필요 있으면 `onInput(event)` 문서화.

5. variant/size 등 상태별 props

- 미리 정의된 문자열 리터럴 집합 사용(예: `'primary'|'secondary'`), Theme token에 매핑.
- CSS 클래스 매핑 테이블을 내부에 뒤 일관성 유지.

6. Polymorphic / as prop

- 요소 태그를 바꿀 필요 있으면 `as` prop 제공. (링크, 라우터 `Link` 로 바꾸기 등)
- TS에서는 제네릭으로 안전하게 타입 처리 권장(간단 예제 아래 제공).

7. Pass-through / Prop forwarding

- `...rest` 로 root 엘리먼트에 전달할 때는 내부 전용 props(예: `variant`)를 제외하고 안전히 스프레드.
- DOM에 불필요한 props(예: `variant`)가 넘어가지 않도록 필터링.

기본값 설정 (JS / TS)

- 함수형 컴포넌트에서 ES6 기본값 권장:

```
function Button({ variant = 'primary', size = 'md', disabled = false }) { ... }
```

- PropTypes (JS 프로젝트):

```
Button.propTypes = { variant: PropTypes.oneOf(['primary', 'secondary']) };
```

- TypeScript 인터페이스:

```
type ButtonProps = { variant?: 'primary'|'secondary'; size?: 'sm'|'md'|'lg'; disabled?: boolean; };
```

권장 콜백 서명(예)

- 간단한 값 전달: `onChange(value: string): void`
- 아이템 선택: `onSelect(itemId: string): void`

- 전체 이벤트 필요 시: `onInput(e: React.ChangeEvent<HTMLInputElement>): void` (문서화 필요)

참조 안정성(성능)

- 부모에서 객체/함수 props를 inline으로 생성하면 자식이 매 렌더마다 재렌더됩니다.
- 해결: `useCallback`, `useMemo` 로 안정화.

```
const handleClick = useCallback(() => setOpen(true), [setOpen]);
const options = useMemo(() => ({ a: 1 }), []);
<Comp onClick={handleClick} options={options} />
```

Prop 확장/옵션 설계 팁

- 너무 많은 boolean props는 API 복잡도를 키움 → `variant` 로 묶거나 옵션 객체 고려.
- 하지만 옵션 객체를 남발하면 가독성 및 자동 문서화 툴(Storybook controls)에서 불리함 — 균형 필요.

② Props 전달 계층 최소화

props drilling을 최소화하고 Context API나 Zustand 등 전역 상태를 활용해 하위 컴포넌트에 필요한 데이터만 전달한다.

해결 전략(우선순위)

1. 상태 위치 재설계

- 상태는 '누가 그 값을 실제로 사용하는가' 기준으로 올립니다. 공통으로 사용하면 상위로, 한 곳에서만 사용하면 더 로컬로.

2. Composition(컴포지션)

- 필요한 UI를 `children` / `render-prop` 으로 위임해 중간 전달 생략.

3. Context API

- 전역적 설정(Theme, Locale, Auth 등)에는 Context 사용.
- 주의: 빈번히 변경되는 값(예: 실시간 카운트)을 Context로 두면 많은 컨슈머가 재렌더 → Context 분리/분할 필요.
- 메모이제이션: Provider가 전달하는 value는 `useMemo` 로 감싸서 값 참조 안정화.

```
const ThemeContext = createContext({ theme: 'light' });
function ThemeProvider({children}){
  const [theme, setTheme] = useState('light');
  const value = useMemo(() => ({theme, setTheme}), [theme]);
  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}
```

4. 전역 상태 라이브러리 (Zustand 등)

- Zustand: selector를 사용하면 필요한 slice만 구독해 재렌더를 국한시킬 수 있어 prop drilling 대체로 좋음.

```
import create from 'zustand';
export const useStore = create(set => ({ token: null, setToken: t => set({ token: t }) }));
// 사용
const token = useStore(state => state.token);
```

5. useContextSelector / use-subscription

- 선택적인 업데이트(consumer가 바뀐 값만 구독) 구현 가능(추가 라이브러리).

실무 권장 패턴

- 설정/읽기 전용 값(Theme, i18n, config) → Context
- 빈번 변경/도메인 상태(카드, 인증 상태) → 전역 스토어(Zustand/Redux) + selector
- 소규모 전달 → 단순 props
- 중간 컴포넌트가 단지 전달자라면 컴포지션 재설계 고려

③ 폴더 구조 설계

도메인별 또는 기능별 폴더 구조를 적용하고 components/common, components/domain, pages, styles 등으로 구성해 관리 효율을 높인다.

고려사항

- 프로젝트 규모(작음→작은 규칙, 큼→엄격한 규칙)
- 팀 업무 방식(디자이너와 협업? Storybook 중시?)
- 재사용성·테스트·배포(모노레포?) 요구사항

도메인(Feature/Domain) 기반 구조

장점: 유지보수(도메인 단위 배포), 단점: UI 재사용 탐색 어려움

```
src/
├─ features/
│  └─ auth/
│     ├── components/
│     ├── hooks/
│     └─ pages/
└─ products/
   ├── components/
   └─ services/
└─ shared/
   ├── ui/
   └─ utils/
```

기능(UI) 기반 구조

장점: 재사용성, Storybook 친화적, 단점: 도메인별 파일 산재 가능

```
src/
├─ components/
│  └─ common/
│     ├── Button/
│     │  ├── Button.jsx
│     │  ├── Button.module.css
│     │  └─ Button.test.jsx
│     └─ index.js
└─ layout/
└─ pages/
└─ hooks/
```

혼합형(Real-world 추천)

도메인별 폴더 + 공통 UI 디렉터리

```
src/
├─ components/      # 공통 UI
│  ├── Button/
│  ├── Input/
│  └─ index.js      # barrel
└─ features/
```

```

|   |─ Cart/
|   |─ Checkout/
|   └─ pages/
|   └─ styles/
|   └─ lib/           # api, store, utils

```

컴포넌트 폴더 템플릿(권장)

```

Button/
├─ Button.jsx        # component 구현 (PascalCase)
├─ Button.module.css # 혹은 Button.styles.js (styled-components)
├─ Button.test.jsx
├─ Button.stories.jsx
├─ types.ts          # (TS 프로젝트)
└─ index.js          # export { default } from './Button';

```

Barrel file (index.js) 패턴

- `components/index.js` :

```

export { default as Button } from './Button';
export { default as Input } from './Input';

```

- 사용

```

import { Button, Input } from 'src/components';

```

④ 컴포넌트 파일 명명 규칙

PascalCase로 컴포넌트 이름을 명시하고, index.js를 통한 재export 구조로 사용성을 높이며 역할과 기능을 명확히 표시한다.

파일명 & 컴포넌트명

- **PascalCase** 사용: `Button.jsx`, `UserCard.jsx`
- 컴포넌트명과 파일명 일치: `export default function Button() {}`

export 스타일

- 추천: **default export** 한 가지 방식으로 파일당 하나의 컴포넌트 (팀에 따라 named export 선호 가능 — 일관성 유지가 관건)
- 파일 내부에서 named export 보조(예: 타입, helper) 가능:

```

export function Button(...) { ... }
export default Button;

```

index.js 재export

- 각 컴포넌트 폴더에 `index.js` 를 두어 루트에서 간편하게 import 가능하게 함.

```

// components/Button/index.js
export { default } from './Button';

```

테스트/스토리/스타일 공존 규칙

- 같은 폴더에 `.test` / `.stories` / `.module.css` 를 함께 두어 관련 리소스 묶음화.

기타 네이밍 컨벤션

- boolean props: `is`, `has` 접두사(예: `isActive`)
- event props: `on` 접두사(예: `onClick`)
- handlers in parent: `handle` 접두사(예: `handleSubmit`)

(4) 실무 관점의 설계 사례 비교

실무 관점의 설계 사례 비교는 도메인 중심, 기능 중심, 혼합형 설계를 비교하며 유지보수성, 재사용성, 테스트 용이성, 협업 효율성을 기준으로 장단점을 평가하고, 팀 규모와 프로젝트 특성에 맞는 최적 설계 전략을 선택하는데 도움을 준다.

① 도메인 중심 설계

서비스나 기능 단위로 폴더와 컴포넌트를 그룹화하여 대규모 프로젝트에서 유지보수와 관리 효율성을 높인다.

서비스의 **비즈니스 도메인**(예: `auth`, `cart`, `checkout`, `products`) 별로 폴더와 모듈을 묶는 방식. 각 도메인이 자체 컴포넌트, 혹은 서비스(API), 테스트를 가진다.

예시 폴더 구조

```
src/
├── features/
│   ├── auth/
│   │   ├── components/
│   │   │   └── LoginForm.jsx
│   │   ├── hooks/
│   │   │   └── useAuth.js
│   │   ├── api/
│   │   │   └── authSlice.js
│   └── products/
│       ├── components/
│       │   └── ProductList.jsx
│       ├── pages/
│       └── services/
└── shared/      # 아주 최소한의 공통(ui, utils)
    ├── ui/
    └── utils/
```

장점

- 도메인 책임이 명확해 팀/기능 단위로 작업 분리(팀 소유권 설정) 용이.
- 도메인 단위로 빌드·테스트·배포(마이크로 프론트엔드 또는 패키지화) 가능.
- 대규모 코드베이스에서 **변경 영향 범위 파악**이 쉽다.

단점

- 공통 UI(버튼/폼 등)가 여러 도메인에 복제/중복될 가능성.
- 재사용성 높은 UI를 찾기 어렵고 Storybook 같은 도구로 문서화하기 번거로울 수 있음.
- 초기 구조 설계·규칙 합의 비용이 큼.

적합한 상황

- 대규모 팀(도메인별 팀) / 복잡한 비즈니스 로직 / 마이크로서비스 연계 프로젝트.

운영 권장

- `shared/ui` 에 최소 공통 컴포넌트 모음(버튼 등)을 두고, 점진적으로 확장.

- 도메인 내부 테스트·스토리 우선, 공통화가 필요하면 `shared` 로 이동(리팩토링 정책 수립).

② 기능 중심 설계

Button, Card, Input 등 UI 단위로 폴더를 구성하여 재사용성을 높이고 Storybook과 테스트 환경에서 활용하기 용이하다.

UI 단위(버튼, 카드, 입력 등)를 **기능별로 재사용 가능한 컴포넌트** 중심으로 모아 두는 방식. UI 라이브러리/디자인 시스템을 만들기 쉬움.

예시 폴더 구조

```
src/
├── components/
│   ├── Button/
│   │   ├── Button.jsx
│   │   ├── Button.module.css
│   │   ├── Button.stories.jsx
│   │   └── index.js
│   ├── Input/
│   └── Card/
├── pages/
└── features/ # 도메인별 로직은 페이지/feature로 분리
```

장점

- 재사용성과 일관성 극대화 — Storybook으로 문서화하기 쉽다.
- 디자이너·프론트엔드가 공통 컴포넌트를 빠르게 탐색·사용 가능.
- UI 라이브러리/패턴(variant, size, as 등)을 표준화하기 용이.

단점

- 도메인 특화 로직(서비스 연동 등)이 컴포넌트에 섞이면 역효과.
- 작은 프로젝트에서는 구조가 과도할 수 있음(관리 오버헤드).

적합한 상황

- 중대형 프로젝트에서 일관된 디자인 시스템이 필요할 때, 제품 군이 확장될 때, 외부/내부에 UI 라이브러리를 배포하려 할 때.

운영 권장

- Storybook + Chromatic(시각 회귀) 사용.
- Components 디렉터리를 독립 패키지(모노레포)로 분리하여 재사용·버전 관리 가능(예: `packages/ui-button`).

③ 혼합형 설계

도메인별 구조에 공통 컴포넌트 모듈을 포함하여 실무 프로젝트에서 유지보수성과 재사용성을 동시에 확보한다.

도메인 중심 구조를 기본으로 하되, 공통·범용 컴포넌트는 별도의 `components` (또는 `shared/ui`)로 분리하는 방식. 실무에서 가장 자주 쓰이는 패턴.

예시 폴더 구조

```
src/
├── components/ # 공통 UI (Button, Input, Card)
├── features/
│   ├── auth/
│   └── products/
├── pages/
├── lib/ # api, utils, hooks
└── styles/
```

장점

- 도메인별 책임 분리 + 재사용 가능한 UI 확보라는 장점 동시 획득.

- 확장성·유지보수성의 균형이 잘 맞음.
- 팀 단위로 작업 소유권을 정하고 공통 컴포넌트는 중앙에서 관리 가능.

단점

- 경계(무엇을 components로 둘지)에 대한 합의가 필요하며, 초기 합의가 없으면 혼란 발생.
- 공통 컴포넌트가 도메인 종속으로 변질될 위험(강한 규칙 필요).

적합한 상황

- 실무(중간/대형) 프로젝트에서 가장 널리 추천되는 접근. 팀이 성장해도 유연하게 대응 가능.

운영 권장

- 명확한 **소유권 규칙**(예: components/* 은 UI팀 소유, features/* 은 도메인팀 소유) 설정.
- 변경 시 API 안정성(semver)·릴리즈 노트 요구.
- Storybook으로 components 공개, 테스트 케이스 및 접근성 체크 필수.

④ 설계 기준 비교 표

유지보수성, 재사용성, 테스트 용이성, 협업 효율성을 기준으로 설계 사례를 비교해 프로젝트 상황에 맞는 최적의 구조를 결정한다.

설계 방식	유지보수성	재사용성	테스트 용이성	협업 효율	초기 설정 비용	확장성(팀/프로젝트)	권장 팀 규
도메인 중심	높음(도메인 단위)	중	쉽(도메인 단위 테스트)	팀별 소유로 효율적	중간~높음	높음(대규모)	대규모/여
기능 중심	중	높음(디자인 시스템)	매우 용이(Storybook)	디자이너·프론트 협업 우수	중간	중~높음	중형 이상 러리 목적
혼합형	높음(균형형)	높음	용이	균형적	중간	매우 유연	중~대형(; 장)